

Documentation Development Lifecycle

**A repeatable, SME-driven
workflow for producing
accurate technical
documentation**

Overview

This document describes the end-to-end process I follow when producing technical documentation – from initial request through

publication and long-term maintenance. It's built around a simple principle: **subject matter experts (SMEs) supply the technical truth; I supply the structure, clarity, and quality control; AI tools accelerate drafting without replacing judgment or verification.**

This isn't a workflow centered on AI. It's a workflow centered on people, with AI used deliberately at one stage — drafting — inside a process that still depends on human expertise at every point that matters.

Workflow at a Glance

flowchart LR

A[Documentation
Request]

--> B[SME
Preparation]

B -->

C[SME
Interview]

C --> D[Organize &

Research]

D --> E[AI-
Assisted
Draft]

E -->

F[Technical
Review]

F --> G[Edit]

G --> H[Publish]

H --> I[Maintain]

F -.needs correction.->

C

1. Define the Documentation Request

Before any writing starts, get clear answers to:

- **What** needs to be documented?
- **Who** is the audience?
- **What** is the deliverable format (user guide, SOP, API reference, training manual)?
- **When** is it due?

A vague request produces a vague document. This step sets scope before it can drift.

2. Prepare the SME

Good interviews are won before they start.

- Send the [SME interview questionnaire](#) in advance.
- Explain the meeting's goals so the SME knows what "useful" looks like.
- Ask the SME to gather diagrams, screenshots, configuration files, or any existing (even outdated) documentation.

This matters because it shifts the meeting from *recall* to *explanation* – the SME spends the time walking through what they know instead of trying to remember it on the spot.

3. Interview the SME

- Record the meeting, with permission.
- Ask follow-up questions in the moment rather than saving them for later.
- Capture exact terminology as the SME uses it.
- Actively listen for gaps — what's being assumed but not said.

4. Organize the Material

Before drafting, sort everything gathered into a consistent structure:

Category	Captures
Purpose	Why this document

Category	Captures
	exists, what problem it solves
Audience	Who reads it, and what they already know
Prerequisites	Access, tools, conditions required before starting
Procedures	The actual steps, in order
Warnings	Safety, compliance, and failure-point information

Category	Captures
Troubleshooting	Common issues and how to resolve them
Verification	How the reader confirms success
References	Related documents, standards, source material

Organizing first — before writing a single sentence of the draft — keeps the draft from being written in the order the SME happened to talk.

5. Draft the Document

This is where AI assistance (including Claude) is genuinely useful, and where its role has to stay bounded correctly: **the SME supplies the expertise; AI helps organize and draft it — it does not invent technical content.**

In practice, this means every fact, step, spec, and warning in the draft traces back to the interview, the SME's source material, or verified reference documentation — never to an AI-generated guess.

6. Technical Review

The draft goes back to the SME with specific questions:

- Is anything technically incorrect?
- Is anything missing?
- Is anything outdated?
- Would you change the order?
- Would you explain anything differently?

This step exists to catch what drafting alone can't: whether the content is *true*, not just whether it reads well.

7. Edit

This is where technical writing craft does the most work. Editing checks for:

- Clarity
- Consistency
- Grammar
- Formatting
- Accessibility
- Style
- Active voice
- Terminology

8. Publish

Convert and deliver the final document to wherever it needs to live:

- Word
- PDF
- HTML
- Markdown

- Knowledge base
- GitHub

The right format depends entirely on the audience and the platform — not on which format was easiest to draft in.

9. Maintain

Documentation isn't finished when it's published. Ongoing maintenance includes:

- Version control
- Revision history
- Periodic review cycles
- Updates when the underlying product or process changes

New technical writers often treat publication as the finish line. It's a checkpoint — the document keeps a maintenance obligation for as long as it stays in use.

Why This Process Matters

Read the sequence again:

1. SME provides expertise.
2. I ask good questions.
3. I organize the information.
4. AI helps produce a draft.
5. I edit.
6. SME validates.
7. I publish and maintain.

At every stage where accuracy is on the line — providing the facts, validating the draft, final quality control — a human is responsible. AI is positioned exactly where it's useful: accelerating the mechanical work of turning organized, verified information into a well-structured first draft.

That's the standard I hold every document in this portfolio to.